INSTITUTE OF TECHNOLOGY OF CAMBODIA
DEPARTMENT OF APPLY MATHEMATICS AND STATISTICS

Chapter-02



UNSLOTH



Presented By: Mr. Vann Visal

Content



1. Introduction to Unsloth
2. Installation Requirements
3. Recommended Installation Method
4. Customizable Chat Templates
5. Benefits
6. Fine-Tuning
7. Model & Preprocessor Setup
8. Data Preparation
9. Supervised Trainer Setup
10. Training Diagnosis
11. Model Checkpoints

1. Introduction to Unsloth



Unsloth is a platform designed to facilitate the fine-tuning of large language models, such as Llama-3, Mistral, Phi-4, and Gemma. It aims to make this process more efficient by reducing memory usage by 70% and accelerating the training process without compromising accuracy.

What is Unsloth?

- Unsloth is a Python library that helps fine-tune large language models efficiently.
- It supports models like Llama-3, Mistral, Phi-4, and Gemma.
- Optimized for reducing memory usage by 70% while maintaining accuracy.

Why Use Unsloth?

- Faster training: Optimized for efficiency.
- Less memory usage: Works well on devices with limited GPU power.
- High accuracy: Achieves state-of-the-art fine-tuning results.

2. Installation Requirements



Installation Requirements detail

Before Installing Unsloth, Ensure:

1. **Operating System Compatibility**
- Works on Linux, Windows, MacOS, Google Colab, and Kaggle.
2. **Python Version**
- Requires Python 3.9 or later.
3. **GPU Support (Optional but recommended)**
- Supports NVIDIA CUDA GPUs for faster performance.
- Can also work on CPU but will be slower.
4. **Pip Installed**
- Ensure that pip (Python package manager) is updated

pip install --upgrade pip

3. Recommended Installation Method



Installation detail

Installing Unsloth Using Pip (Recommended)

1. Open Terminal/Command Prompt.

- Run the installation command:

  pip install unsloth

2. Verify installation:

- Run Python and import Unsloth:

  import unsloth

  print(unsloth.__version__)

- 3. Import Errors
  - Problem: "ModuleNotFoundError: No module named 'unsloth'"
  - Solution: Reinstall Unsloth and check the Python path:

    pip uninstall unsloth

    pip install unsloth

4.Customizable Chat Templates



Customizable Chat Templates detail

Example : Llama-3 template

```
chat_template = """<|begin_of_text|><|start_header_id|>system<|end_header_id|>

{SYSTEM}<|eot_id|><|start_header_id|>user<|end_header_id|>

{INPUT}<|eot_id|><|start_header_id|>assistant<|end_header_id|>

{OUTPUT}<|eot_id|>"""
```

Key Points:

- Customization: Chat templates can be modified to fit different AI models.

Example (Llama-3 template):

- Uses placeholders like {SYSTEM}, {INPUT}, and {OUTPUT}.
- Defines headers for system, user, and assistant roles.
- Utilizes special tokens like <|begin_of_text|>, <|eot_id|>, <|start_header_id|>, and <|end_header_id|> for structured interactions.

5. Benefits

- Task-Specific Adaptation – Models improve accuracy in specialized areas.
- Resource Efficiency – Optimized training reduces GPU/memory usage.
- Improved Performance – Enhanced output relevance and precision.

Code Snippet Explanation

- The code demonstrates configuring a vision model for fine-tuning using Unsloth.
- Fine-tuning options include vision layers, language layers, attention modules, and MLP modules, all enabled (True) by default.
- LoRA Parameters:

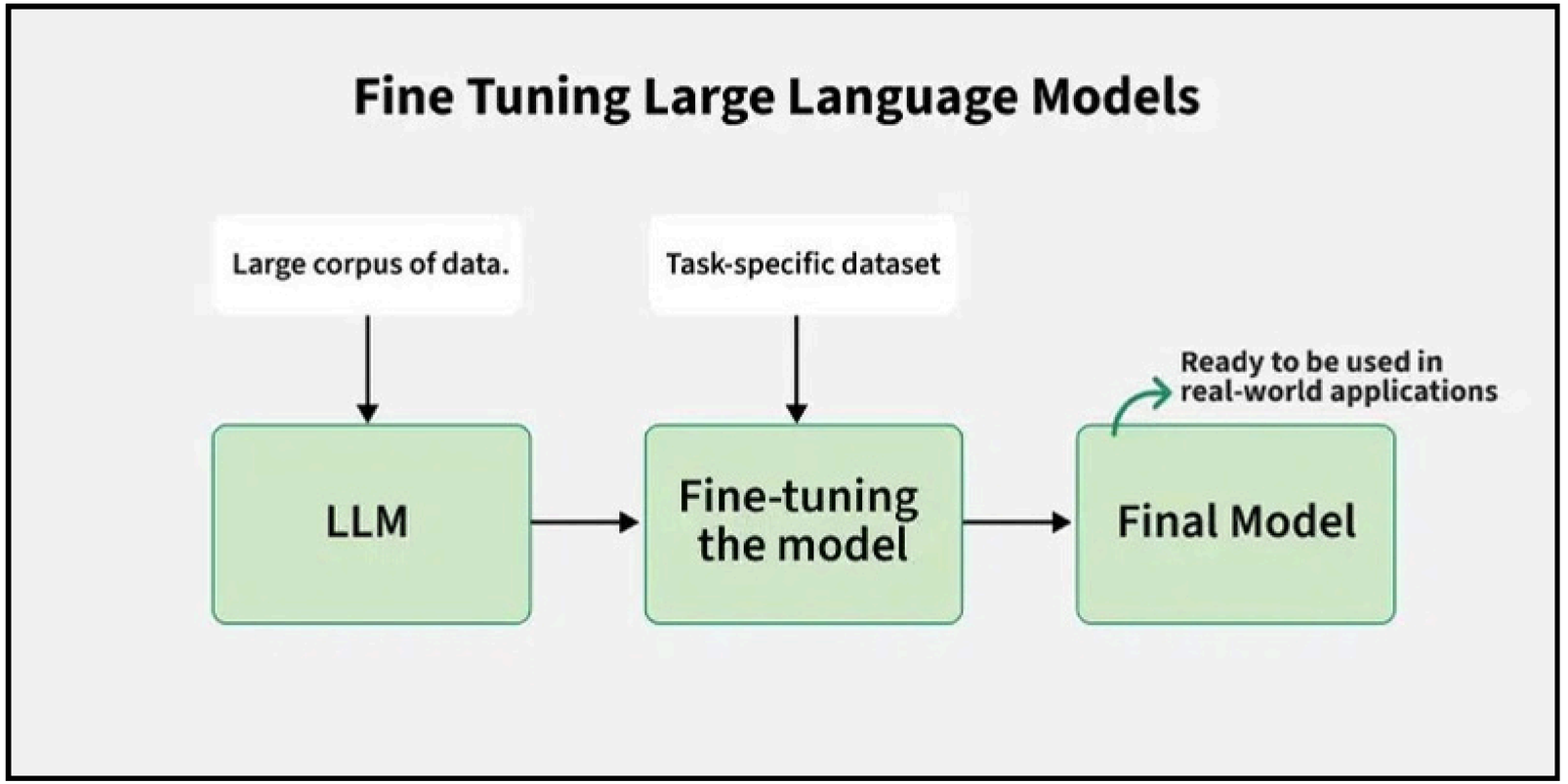
1. r=16 (rank for fine-tuning)
2. lora_alpha=16 (recommended setting)
3. use_rslora=False (stabilized LoRA support)

```
model = FastVisionModel.get_peft_model(
    model,
    finetune_vision_layers     = True, # Set False if not finetuning vision part
    finetune_language_layers   = True, # Set False if not finetuning language part
    finetune_attention_modules = True, # Set False if not finetuning attention layers
    finetune_mlp_modules       = True, # Set False if not finetuning MLP layers

    r = 16,           # The larger, the higher the accuracy, but might overfit
    lora_alpha = 16,  # Recommended alpha == r at least
    lora_dropout = 0,
    bias = "none",
    random_state = 3407,
    use_rslora = False,  # We support rank stabilized LoRA
    loftq_config = None, # And LoftQ
    # target_modules = "all-linear", # Optional now! Can specify a list if needed
)
```

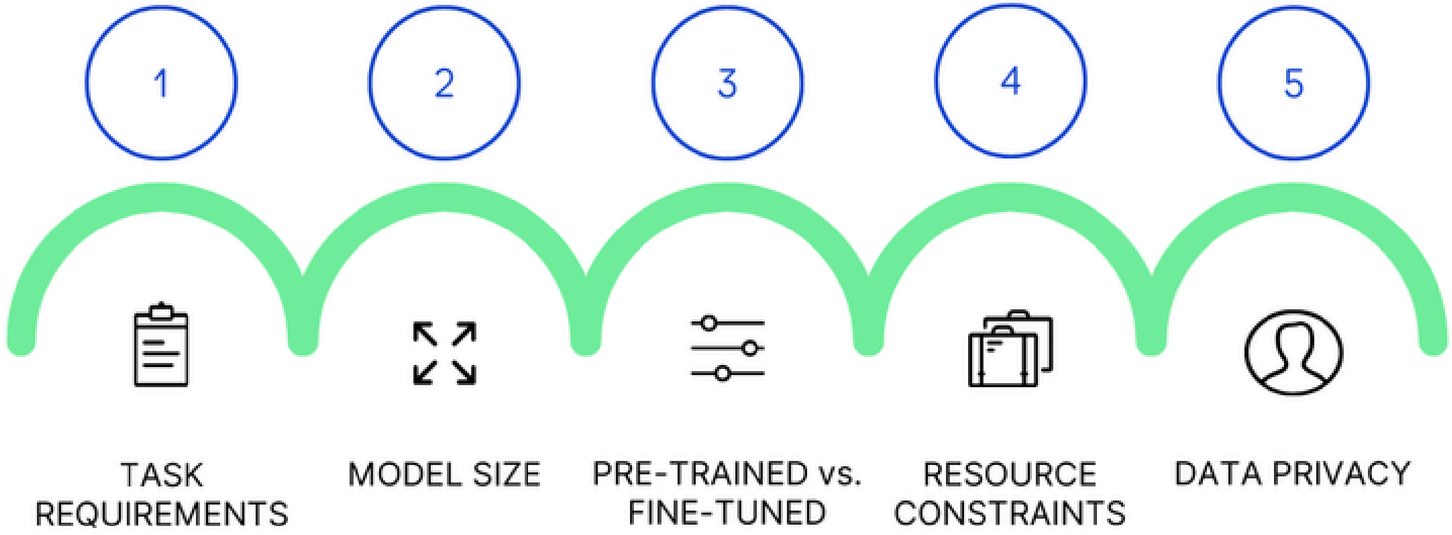
6. Fine-Tuning

Fine-tuning is a technique where a pretrained model is adapted to a new, specific task by continuing its training on a smaller, task-specific dataset. It leverages the knowledge learned from large-scale datasets and adjusts the model's parameters to improve performance on the new task.



7. Model & Preprocessor Setup

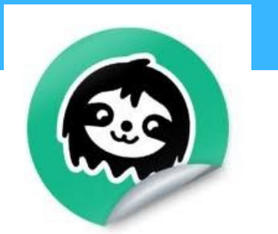
Model choosing



7. Model & Preprocessor Setup



A model loader is a utility or function that allows you to load a pretrained model, its architecture, and the associated weights (parameters) into memory so that you can use it for inference or further training (fine-tuning).







INSTITUTE OF TECHNOLOGY OF CAMBODIA
DEPARTMENT OF APPLY MATHEMATICS AND STATISTICS

Chapter-03

Gemma 3

Presented By: Mr. Vann Visal

What is Gemma 3?

Gemma 3 is Google's latest iteration of open weight LLMs. It comes in four sizes, **1 billion, 4 billion, 12 billion, and 27 billion** parameters with base (pre-trained) and instruction-tuned versions. Gemma 3 goes multimodal ! The 4, 12, and 27 billion parameter models can process both images and text, while the 1B variant is text only.

Pre Trained	Instruction Tuned	Multimodal	Multilingual	Input Context Window
gemma-3-1b-pt	gemma-3-1b-it	❌	English	32K
gemma-3-4b-pt	gemma-3-4b-it	✅	+140 languages	128K
gemma-3-12b-pt	gemma-3-12b-it	✅	+140 languages	128K
gemma-3-27b-pt	gemma-3-27b-it	✅	+140 languages	128K

Gemma evaluation

Perplexity:

Definition: Measures how well a model predicts the next word in a sentence. Lower perplexity means the model is better at understanding the structure of language.

Calculation Process:

- Tokenize the input text into words or subwords.
- Compute the log probability of each word.
- Average the log probabilities over the sequence.
- Compute the final perplexity score.

Gemma evaluation

BLEU (Bilingual Evaluation Understudy)

Definition: Compares the generated command with a reference command to measure similarity.

Calculation Process:

- Tokenize the reference and generated text.
- Count the overlapping n-grams.
- Compute n-gram precision for different values of n (1 to 4).
- Apply the brevity penalty (BP) if the generated text is too short.
- Calculate the final BLEU score.

Output Values

- bleu (float): bleu score
- precisions (list of floats): geometric mean of n-gram precisions,
- brevity_penalty (float): brevity penalty,
- length_ratio (float): ratio of lengths,
- translation_length (int): translation_length,
- reference_length (int): reference_length

Gemma evaluation

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE measures how well the generated text matches the reference text using precision, recall, and F1-score. It is commonly used for summarization tasks.

Calculation Process:

- Tokenize the reference and generated text.
- Compute n-gram matches.
- Calculate Precision, Recall, and F1-score.
- Compute ROUGE-N and ROUGE-L scores.

!pip install rouge-score

ROUGE Variants:

- ROUGE-N: Measures n-gram overlap (e.g., ROUGE-1 for unigrams, ROUGE-2 for bigrams).
- ROUGE-L: Measures longest common subsequence (LCS).
- ROUGE-W: Weighted LCS.

Gemma 2 evaluation

Evaluation Type	Metric and Purpose
Language Fluency	Perplexity (Lower scores indicate better predictions)
Content Generation	ROUGE, BLEU (Compare generated text to reference texts)

Key Metrics Recap

BLEU	Focuses on precision, often used for evaluating machine translation by comparing generated text to reference translations.
ROUGE	Emphasizes recall, commonly applied in summarization tasks to measure how well the generated output captures key content.

What is Temperature in LLMs?

The temperature parameter controls the randomness of the text generated by a language model.

- Lower temperature (e.g., 0.1 - 0.5): Makes output more deterministic and predictable.
- Higher temperature (e.g., 0.8 - 1.5): Makes output more diverse and creative but also more random.

T (temperature)adjusts probability distribution:

- If T → 0, the model chooses the most likely word.
- If T → 1, normal probability distribution is used.
- If T > 1, probabilities become more evenly spread, increasing randomness.

```
# Applying temperature parameter
outputs = model.generate(
    input_ids=inputs,
    max_new_tokens=64,
    use_cache=True,
    temperature=0.7  # Adjust randomness of output
)
```

Recommended Setting for Robotic Commands: temperature=0.3 to 0.7 (for precise but flexible responses).



Thank You!

